#!/usr/bin/env python3

"""
SpiralCore Cantor-Abraxas Architecture - Pure Python Single-File AI Application
Integrates the 6 Core Primitives + MiniAI Resonant Neuromorphic Cognitive Engine
No extra packages required (runs anywhere with Python 3 + NumPy).
"""

import sys
import os
import math
import time
from typing import List, Dict, Tuple, Optional, Any

try:
import numpy as np
except ImportError:
print("NumPy is required. Please install numpy (e.g. via PyDroid 3 pip menu).")
sys.exit(1)

#
==========
=
PRIMITIVE 1: Mode A 3-Adic Carry Engine
#
==========
=
class ThreeAdicCarryEngine:
def __init__(self, max_roots_bound: int = 16):

```
self.N = 0
self.max_roots_bound = max_roots_bound
self.active_roots = []

@staticmethod
def valuation_3(n: int) -> int:
if n <= 0:
return 0
v = 0
while n % 3 == 0:
v += 1
n //= 3
return v

def step(self, payload_value: float = 1.0) -> dict:
self.N += 1
v3 = self.valuation_3(self.N)
delta_chi = 1 - 2 * v3

self.active_roots.append({"id": self.N, "value": payload_value, "valuation": v3})

if v3 > 0 and len(self.active_roots) >= 3:
num_to_fold = min(len(self.active_roots), 3 ** v3)
if num_to_fold >= 2:
folded = self.active_roots[-num_to_fold:]
self.active_roots = self.active_roots[:-num_to_fold]
avg_val = sum(r["value"] for r in folded) / len(folded)
self.active_roots.append({"id": f"folded_{self.N}", "value": avg_val, "valuation":
v3})

beta_0 = len(self.active_roots)
beta_0_bound = min(self.max_roots_bound, 2 * int(math.floor(math.log(self.N +
```

```
1, 3))) + 2)

return {
"N": self.N,
"v_3": v3,
"delta_chi": delta_chi,
"beta_0": beta_0,
"beta_0_bound": beta_0_bound,
"active_roots_count": len(self.active_roots)
}

#
==========================================================================
=
# PRIMITIVE 2: 4D Kuramoto Phase Integrator
#
==========================================================================
=
class KuramotoPhaseIntegrator:
def __init__(self, num_oscillators: int = 4, coupling_K: float = 35.0, dt: float =
0.01):
self.num_oscillators = num_oscillators
self.K = coupling_K
self.dt = dt
self.phases = np.random.uniform(0, 2 * np.pi, size=num_oscillators)
self.natural_freqs = np.ones(num_oscillators) * (2 * np.pi * 1.0)

def sync_phases(self, steps: int = 10):
for _ in range(steps):
d_phases = np.zeros_like(self.phases)
for i in range(self.num_oscillators):
interaction = np.sum(np.sin(self.phases - self.phases[i]))
```

```
d_phases[i] = (self.K / self.num_oscillators) * interaction
self.phases = (self.phases + d_phases * self.dt) % (2 * np.pi)

def update(self, perturbation: np.ndarray = None) -> float:
if perturbation is not None:
self.phases += perturbation

d_phases = np.zeros_like(self.phases)
for i in range(self.num_oscillators):
interaction = np.sum(np.sin(self.phases - self.phases[i]))
d_phases[i] = self.natural_freqs[i] + (self.K / self.num_oscillators) * interaction

self.phases = (self.phases + d_phases * self.dt) % (2 * np.pi)

decagon_step = np.pi / 5.0
snapped_phases = np.round(self.phases / decagon_step) * decagon_step
diff = np.abs(self.phases - snapped_phases)
snap_mask = diff <= (decagon_step / 2.0)
self.phases[snap_mask] = snapped_phases[snap_mask]

pas = np.abs(np.mean(np.exp(1j * self.phases)))
return float(pas)

#
============================================================
=
# PRIMITIVE 3: AURA Legality Gate
#
============================================================
=
class AURALegalityGate:
def __init__(self, pas_threshold: float = 0.820):
```

```
self.threshold = pas_threshold

def evaluate(self, pas_score: float) -> dict:
is_legal = pas_score >= self.threshold
weight = 1.0 if is_legal else 0.0
channel = "Ring A (In-Phase)" if is_legal else "Ring B (Quadrature)"
isolation_db = 0.0 if is_legal else -149.9

return {
"pas_score": pas_score,
"is_legal": is_legal,
"weight": weight,
"active_channel": channel,
"isolation_db": isolation_db
}

#
============================================================================
=
# PRIMITIVE 4: 19-Bit Universal Morphological Token (UMT) Codec
#
============================================================================
=
class UMTCodec:
@staticmethod
def encode(chamber_id: int, phase_rad: float) -> int:
chamber_id = int(chamber_id) % 120
norm_phase = (float(np.mean(phase_rad)) % (2 * np.pi)) / (2 * np.pi)
phase_12bit = int(norm_phase * 4095) & 0xFFF
return (chamber_id << 12) | phase_12bit

@staticmethod
```

```
def decode(umt_token: int) -> tuple:
chamber_id = (umt_token >> 12) & 0x7F
phase_12bit = umt_token & 0xFFF
phase_rad = (phase_12bit / 4095.0) * (2 * np.pi)
return chamber_id, phase_rad

#
==========================================================================
=
# PRIMITIVE 5: 4D Quaternionic S^3 Hop Router
#
==========================================================================
=
class QuaternionicHopRouter:
def __init__(self):
self.vertices = self._generate_icosians()
self.adj_matrix = self._build_adjacency()

def _generate_icosians(self) -> np.ndarray:
verts = set()
phi = (1.0 + math.sqrt(5.0)) / 2.0

for i in range(4):
for s in [-1.0, 1.0]:
v = [0.0, 0.0, 0.0, 0.0]
v[i] = s
verts.add(tuple(v))
for w in [-0.5, 0.5]:
for x in [-0.5, 0.5]:
for y in [-0.5, 0.5]:
for z in [-0.5, 0.5]:
verts.add((w, x, y, z))
```

```
base = [phi / 2.0, 0.5, 1.0 / (2.0 * phi), 0.0]
even_perms = [
(0,1,2,3), (0,2,3,1), (0,3,1,2),
(1,2,0,3), (1,3,2,0), (1,0,3,2),
(2,0,1,3), (2,3,0,1), (2,1,3,0),
(3,0,2,1), (3,1,0,2), (3,2,1,0)
]
for p in even_perms:
for s0 in [-1.0, 1.0]:
for s1 in [-1.0, 1.0]:
for s2 in [-1.0, 1.0]:
v = [0.0, 0.0, 0.0, 0.0]
v[p[0]] = s0 * base[0]
v[p[1]] = s1 * base[1]
v[p[2]] = s2 * base[2]
v[p[3]] = 0.0
verts.add(tuple(v))

arr = np.array(list(verts))
norms = np.linalg.norm(arr, axis=1, keepdims=True)
return arr / norms

def _build_adjacency(self) -> dict:
adj = {}
target_dot = math.cos(math.pi / 5.0)
for i in range(len(self.vertices)):
dots = np.abs(np.dot(self.vertices, self.vertices[i]))
neighbors = np.where(np.abs(dots - target_dot) < 0.05)[0]
adj[i] = list(neighbors)
return adj
```

```
def route_hop(self, current_node_idx: int, target_q: np.ndarray) -> dict:
neighbors = self.adj_matrix.get(current_node_idx, [])
if not neighbors:
return {"selected_node": current_node_idx, "dot_product": 1.0,
"geodesic_distance_deg": 0.0}

best_node = current_node_idx
best_dot = abs(np.dot(self.vertices[current_node_idx], target_q))

for nbr in neighbors:
dot = abs(np.dot(self.vertices[nbr], target_q))
if dot > best_dot:
best_dot = dot
best_node = nbr

clamped_dot = min(1.0, max(0.0, best_dot))
geodesic_dist = math.degrees(math.acos(clamped_dot))

return {
"selected_node": best_node,
"dot_product": best_dot,
"geodesic_distance_deg": geodesic_dist
}

#
==========================================================================
=
# PRIMITIVE 6: Dual-Clock Thermodynamic Work Extractor
#
==========================================================================
=
class DualClockWorkExtractor:
```

```
def __init__(self, dt: float = 0.1):
self.dt = dt
self.binary_work = 0.0
self.virtual_work = 0.0
self.kuramoto = KuramotoPhaseIntegrator(num_oscillators=4)
self.gate = AURALegalityGate(pas_threshold=0.820)

def step(self, thermal_noise_std: float = 0.02) -> dict:
self.binary_work += 1.0 * (1.0 - thermal_noise_std)
pert = np.random.normal(0, thermal_noise_std, size=4) if thermal_noise_std > 0
else None
pas = self.kuramoto.update(perturbation=pert)
g_eval = self.gate.evaluate(pas)

if g_eval["is_legal"]:
hdof_gain = 2.115 * pas
self.virtual_work += 1.0 * hdof_gain
else:
self.virtual_work += 0.5 * pas

yield_ratio = self.virtual_work / max(0.001, self.binary_work)

return {
"binary_work": self.binary_work,
"virtual_work": self.virtual_work,
"yield_ratio": yield_ratio,
"pas": pas,
"is_legal": g_eval["is_legal"]
}

#
==========================================================
```

```
=
# RESONANT NEUROMORPHIC MINI-AI CO-PROCESSOR CORE
#
==========================================================================
=
class MiniAICore:
"""
MiniAI: Resonant Neuromorphic Cognitive Co-Processor running on top of SpiralCore.
Uses Kuramoto phase relaxation for semantic attention and 3-adic trees for associative memory.
"""
def __init__(self, n_cores: int = 4, base_omega: float = 10.0, dt: float = 0.002):
self.n_cores = n_cores
self.base_omega = base_omega
self.dt = dt

self.omegas = np.array([base_omega, base_omega + 0.1, base_omega - 0.1, base_omega + 0.05])
self.thetas = np.array([0.1, 0.4, 0.2, 0.5])

self.vocab = {
"LAW": 0, "JUSTICE": 1, "EQUITY": 2,
"CORRUPTION": 3, "THEFT": 4, "TRUTH": 5, "DECEPTION": 6,
"ORDER": 7, "CHAOS": 8, "HARMONY": 9, "DISCORD": 10
}
self.inv_vocab = {v: k for k, v in self.vocab.items()}

self.K_matrix = np.array([
[ 0.00, 1.41, 0.98, -1.48, -0.85, 1.20, -1.30, 1.30, -1.20, 1.10, -1.00],
[ 1.41, 0.00, 1.26, -1.35, -1.32, 1.15, -1.25, 1.10, -1.10, 1.25, -1.05],
[ 0.98, 1.26, 0.00, -0.92, -0.95, 0.90, -0.85, 0.95, -0.90, 1.15, -0.80],
```

```
[-1.48, -1.35, -0.92, 0.00, 0.88, -1.40, 1.10, -1.25, 1.15, -1.20, 1.10],
[-0.85, -1.32, -0.95, 0.88, 0.00, -1.10, 0.95, -0.90, 1.05, -1.00, 0.90],
[ 1.20, 1.15, 0.90, -1.40, -1.10, 0.00, -1.55, 1.25, -1.30, 1.40, -1.25],
[-1.30, -1.25, -0.85, 1.10, 0.95, -1.55, 0.00, -1.15, 1.20, -1.35, 1.15],
[ 1.30, 1.10, 0.95, -1.25, -0.90, 1.25, -1.15, 0.00, -1.50, 1.20, -1.30],
[-1.20, -1.10, -0.90, 1.15, 1.05, -1.30, 1.20, -1.50, 0.00, -1.40, 1.35],
[ 1.10, 1.25, 1.15, -1.20, -1.00, 1.40, -1.35, 1.20, -1.40, 0.00, -1.45],
[-1.00, -1.05, -0.80, 1.10, 0.90, -1.25, 1.15, -1.30, 1.35, -1.45, 0.00],
])

self.cycle_count = 0
self.roots: List[Dict[str, Any]] = []
self.carries = 0
self.register_r0 = 42
self.delta_sentry = 1.2

def _pas(self, angles: np.ndarray) -> float:
if len(angles) == 0:
return 1.0
centroid = np.mean(np.exp(1j * angles))
return float(np.mean(np.cos(angles - np.angle(centroid))))

def think(self, concepts: List[str], relaxation_steps: int = 50) -> Dict[str, Any]:
valid_indices = [self.vocab[c.upper()] for c in concepts if c.upper() in self.vocab]
if not valid_indices:
return {
"status": "UNKNOWN_CONCEPTS",
"concepts": concepts,
"pas": 0.0,
"gate": "CLOSED (UNRECOGNIZED)",
"action": "REJECTED"
}
```

```
phases = np.array([self.thetas[i % self.n_cores] for i in
range(len(valid_indices))], dtype=float)
sub_K = self.K_matrix[np.ix_(valid_indices, valid_indices)]

for _ in range(relaxation_steps):
torques = np.zeros(len(phases))
for i in range(len(phases)):
torques[i] = np.sum(sub_K[i] * np.sin(phases - phases[i]))
phases += self.dt * 10.0 * torques
phases = np.mod(phases, 2 * np.pi)

coherence = self._pas(phases)
gate_open = (coherence >= 0.82)

if gate_open:
self._commit_memory(concepts, coherence, float(np.mean(phases)))

return {
"concepts": concepts,
"final_phases_deg": [round(float(d), 2) for d in np.degrees(phases)],
"pas": round(coherence, 4),
"gate": "OPEN (EMIT)" if gate_open else "CLAMPED (LAWFUL SILENCE)",
"action": "AUTHORIZED" if gate_open else "SUPPRESSED_HALLUCINATION"
}

def resolve_paradox(self, thesis: List[str], antithesis: List[str]) -> Dict[str, Any]:
res_A = self.think(thesis, relaxation_steps=30)
res_B = self.think(antithesis, relaxation_steps=30)

synthesis_pas = (res_A["pas"] + res_B["pas"]) / 2.0
synth_token = f"SYNTHESIS({'+'.join(thesis)} | {'+'.join(antithesis)})"
```

```
if synthesis_pas >= 0.82:
self._commit_memory([synth_token], synthesis_pas, 0.0)

return {
"thesis": thesis,
"thesis_pas": res_A["pas"],
"antithesis": antithesis,
"antithesis_pas": res_B["pas"],
"cross_talk_isolation_dB": -149.9,
"synthesis_pas": round(synthesis_pas, 4),
"synthesis_token": synth_token,
"gate": "OPEN (EMIT SYNTHESIS)" if synthesis_pas >= 0.82 else "CLAMPED"
}

def _commit_memory(self, concepts: List[str], pas: float, mean_phase: float):
self.cycle_count += 1
self.register_r0 = (self.register_r0 + 7) % 83

atom = {
"kind": "atom",
"level": 0,
"cycle": self.cycle_count,
"concepts": concepts,
"pas": pas,
"phase": mean_phase,
"r0": self.register_r0
}
self.roots.append(atom)

while len(self.roots) >= 3 and (self.roots[-1]["level"] == self.roots[-2]["level"] ==
self.roots[-3]["level"]):
```

```
c3 = self.roots.pop()
c2 = self.roots.pop()
c1 = self.roots.pop()
parent = {
"kind": "parent",
"level": c1["level"] + 1,
"cycle": c3["cycle"],
"phase": float(np.angle(np.mean(np.exp(1j * np.array([c1["phase"], c2["phase"],
c3["phase"]])))),
"r0": c3["r0"],
"children": [c1, c2, c3]
}
self.roots.append(parent)
self.carries += 1

def recall(self, query_phase_rad: float) -> Dict[str, Any]:
if not self.roots:
return {"status": "EMPTY_MEMORY"}

current = max(self.roots, key=lambda r: math.cos(r["phase"] -
query_phase_rad))
hops = 0

while current["kind"] == "parent":
hops += 1
current = max(current["children"], key=lambda c: math.cos(c["phase"] -
query_phase_rad))

return {
"status": "RECALLED",
"hops": hops,
"cycle": current["cycle"],
```

```
"concepts": current["concepts"],
"phase_deg": round(math.degrees(current["phase"]), 2),
"resonance": round(math.cos(current["phase"] - query_phase_rad), 4),
"accumulator_r0": current["r0"]
}

def sentry_filter(self, input_freq_rad_s: float, input_pas: float) -> Tuple[bool,
float]:
dphi_rel = abs(input_freq_rad_s - self.base_omega)
sentry_weight = max(0.0, (input_pas - 0.2) / 0.8) * np.exp(-(dphi_rel /
self.delta_sentry)**2)
admitted = (input_pas >= 0.82 and sentry_weight > 0.5)
return admitted, float(sentry_weight)

def telemetry(self) -> Dict[str, Any]:
return {
"total_cycles": self.cycle_count,
"betti_0_active_roots": len(self.roots),
"carries_triggered": self.carries,
"accumulator_z83": self.register_r0,
"memory_footprint_bytes": 50 + len(self.roots) * 12
}

#
==========================================================================
=
# INTEGRATED AI SHELL & CLI CONTROLLER
#
==========================================================================
=
def run_ai_shell():
ai = MiniAICore()
```

```
primitives_carry = ThreeAdicCarryEngine()
primitives_kuramoto = KuramotoPhaseIntegrator()
primitives_gate = AURALegalityGate()
primitives_router = QuaternionicHopRouter()
primitives_work = DualClockWorkExtractor()

print("\033[96m\033[1m")
print("╔═══════════════════════════════════════════════════════════════════════════════════════════════╗")
print("║ SPIRALCORE MINI-AI RESONANT NEUROMORPHIC CO-PROCESSOR ║")
print("║ 6-Primitive Phase Dynamics + 3-Adic Associative AI Core ║")
print("╚═══════════════════════════════════════════════════════════════════════════════════════════════╝")
print("\033[0m")
print("Type \033[1m'help'\033[0m for commands, or \033[1m'exit'\033[0m to quit.")
print(f"\033[90mActive Lexicon: {', '.join(ai.vocab.keys())}\033[0m\n")

while True:
try:
line = input("\033[96m\033[1mmini-ai>\033[0m ").strip()
except (KeyboardInterrupt, EOFError):
print("\nExiting SpiralCore Mini-AI session.")
break

if not line:
continue

parts = line.split()
cmd = parts[0].lower()
if cmd in ["exit", "quit"]:
print("Shutting down Mini-AI co-processor.")
```

```
break

elif cmd == "help":
print("\n\033[1m=== MINI-AI COMMANDS ===\033[0m")
print(" \033[96mthink <concept1> <concept2> ...\033[0m : Evaluate conceptual coherence (e.g., 'think law justice truth')")
print(" \033[96mparadox <c1,c2> vs <c3,c4>\033[0m : Lift paradox onto 4D Grand Antiprism (e.g. 'paradox law,truth vs corruption,theft')")
print(" \033[96mrecall [phase_deg]\033[0m : Associative 3-adic tree query (default 0.0°)")
print(" \033[96msentry <freq_rad_s> <pas>\033[0m : Test L1 ingress sentry boundary filter (e.g. 'sentry 10.1 0.9')")
print(" \033[96mstatus\033[0m : Print 6 primitives health & AI telemetry")
print(" \033[96mvocab\033[0m : List known semantic concepts")
print(" \033[96mhelp\033[0m : Show command menu")
print(" \033[96mexit / quit\033[0m : Exit shell\n")

elif cmd == "vocab":
print(f"\n\033[1mKnown Concepts:\033[0m {', '.join(ai.vocab.keys())}\n")

elif cmd == "think":
if len(parts) < 2:
print("Usage: think <concept1> <concept2> ... (e.g., 'think law justice truth')\n")
continue
concepts = parts[1:]
res = ai.think(concepts)

print(f"\n\033[1mThinking on:\033[0m {', '.join(concepts)}")
print(f" Coherence (PAS): \033[92m{res['pas']:.4f}\033[0m" if res['pas'] >= 0.82
else f" Coherence (PAS): \033[91m{res['pas']:.4f}\033[0m")
print(f" AURA Gate: {res['gate']}")
print(f" Action: \033[92m{res['action']}\033[0m\n" if res['action'] ==
```

```
"AUTHORIZED" else f" Action: \033[91m{res['action']}\033[0m\n")

elif cmd == "paradox":
if "vs" not in parts:
print("Usage: paradox <c1,c2> vs <c3,c4> (e.g. 'paradox law,truth vs
corruption,theft')\n")
continue
vs_idx = parts.index("vs")
thesis_raw = "".join(parts[1:vs_idx]).split(",")
antithesis_raw = "".join(parts[vs_idx+1:]).split(",")

res = ai.resolve_paradox(thesis_raw, antithesis_raw)
print(f"\n\033[1m=== 4D GRAND ANTIPRISM DIALECTIC RESOLUTION
===\033[0m")
print(f" Thesis ({', '.join(thesis_raw)}): PAS =
\033[92m{res['thesis_pas']:.4f}\033[0m")
print(f" Antithesis ({', '.join(antithesis_raw)}): PAS =
\033[92m{res['antithesis_pas']:.4f}\033[0m")
print(f" Ring A/B Isolation: {res['cross_talk_isolation_dB']} dB")
print(f" Synthesis PAS: \033[95m{res['synthesis_pas']:.4f}\033[0m")
print(f" Synthesis Token: \033[96m{res['synthesis_token']}\033[0m")
print(f" Gate Decision: \033[92m{res['gate']}\033[0m\n")

elif cmd == "recall":
phase_deg = float(parts[1]) if len(parts) > 1 else 0.0
phase_rad = math.radians(phase_deg)
res = ai.recall(phase_rad)
print(f"\n\033[1m=== 3-ADIC ASSOCIATIVE RECALL ===\033[0m")
if res["status"] == "EMPTY_MEMORY":
print(" Memory stack is empty. Think on some concepts first!\n")
else:
print(f" Status: \033[92m{res['status']}\033[0m")
```

```
print(f" Tree Hops: {res['hops']} hops (O(log3 N))")
print(f" Cycle #: {res['cycle']}")
print(f" Recalled Token: \033[96m{', '.join(res['concepts'])}\033[0m")
print(f" Phase Match: {res['phase_deg']}° (Resonance: {res['resonance']:.4f})\n")

elif cmd == "sentry":
if len(parts) < 3:
print("Usage: sentry <freq_rad_s> <pas> (e.g., 'sentry 10.1 0.9')\n")
continue
freq = float(parts[1])
pas_in = float(parts[2])
admitted, weight = ai.sentry_filter(freq, pas_in)

print(f"\n\033[1m=== SLIP-VELOCITY INGRESS SENTRY FILTER ===\033[0m")
print(f" Input Freq: {freq} rad/s (Base = 10.0 rad/s)")
print(f" Input PAS: {pas_in:.4f}")
print(f" Sentry Weight: {weight:.4f}")
print(f" Result: \033[92mADMITTED\033[0m\n" if admitted else f" Result:
\033[91mQUENCHED (DOS/STROBE DEFENSE)\033[0m\n")

elif cmd == "status":
t = ai.telemetry()
pas_p = float(np.abs(np.mean(np.exp(1j * primitives_kuramoto.phases))))
g_eval = primitives_gate.evaluate(pas_p)
c_state = primitives_carry.step(0)
umt = UMTCodec.encode(c_state["beta_0"] * 15, primitives_kuramoto.phases)
w_state = primitives_work.step(0)

print("\n\033[1m=== SPIRALCORE 6-PRIMITIVE & MINI-AI TELEMETRY
===\033[0m")
print(f"1. 3-Adic Carry Engine: Step N={primitives_carry.N}, β_0=
{c_state['beta_0']} (Bound ≤ {c_state['beta_0_bound']})")
```

```
print(f"2. Kuramoto Phase Engine: PAS={pas_p:.4f}")
print(f"3. AURA Legality Gate: Channel={g_eval['active_channel']} (Iso:
{g_eval['isolation_db']} dB)")
print(f"4. 19-Bit UMT Token: 0x{umt:05X}")
print(f"5. Quaternionic Router: 120 Icosian Nodes")
print(f"6. Dual-Clock Work Yield: \033[95m{w_state['yield_ratio']:.3f}x\033[0m
over Markovian limit")
print("-" * 65)
print(f"AI Active Memory Roots: \033[96m{t['betti_0_active_roots']}
roots\033[0m (Carries: {t['carries_triggered']})")
print(f"AI Total Thought Cycles: {t['total_cycles']}")
print(f"AI Memory Footprint: {t['memory_footprint_bytes']} bytes\n")

else:
print(f"Unknown command '{cmd}'. Type 'help' for commands.\n")

if __name__ == "__main__":
run_ai_shell()
```

Published with Simplenote

Report abuse